

Instituto Tecnológico de Costa Rica

Campus Tecnológico Local San Carlos

Curso IC 4700 Lenguajes de programación

Definición de los eventos

Profesor:

Oscar Víquez Acuña

Estudiantes:

Yovelky Delgado Solís

Sebastián Vargas Díaz

II Semestre

Año 2026

1. Concepto fundamental de los eventos

GameStats trabajará con eventos que representan lo que sucede durante una partida. Por ejemplo, un movimiento en ajedrez, un tiro en baloncesto una vuelta completada en una carrera o una pastilla consumida en Pac-Man. Estos registros permitirán conocer qué ocurrió, quién participó y en qué momento, para después calcular estadísticas y analizar situaciones de interés.

El proyecto no contempla desarrollar los videojuegos completos. En su lugar, se utilizará un simulador o archivos de prueba que generen eventos coherentes con cada juego.

1.1. Estructura común

Los eventos se enviarán en formato JSON. Aunque cada juego necesita información diferente, todos compartirán los siguientes campos:

Campo	Tipo	Descripción
eventId	string	Identificador único del evento.
timestamp	datetime	Momento en que ocurrió. En JSON se representará como texto en formato ISO 8601 y UTC.
gameId	string	Identificador del juego: CHESS, BASKETBALL o RACING.
gameVersion	string	Versión del juego o de la simulación.
matchId	string	Identificador de la partida.
sequence	int	Orden del evento dentro de la partida.

playerId	string o null	Identificador del jugador que realizó la acción.
type	string	Tipo de evento registrado.
data	Objeto	Información específica del evento.

Esta estructura permitirá recibir eventos de distintos juegos sin cambiar por completo la forma de procesarlos. Las diferencias se encontrarán principalmente en type y data.

El campo sequence ayudará a ordenar los hechos, incluso cuando varios eventos tengan la misma fecha y hora. Por su parte, eventId permitirá reconocer eventos repetidos y evitar que una acción se contabilice más de una vez.

Las duraciones se guardarán en milisegundos. Todos los campos principales estarán presentes, aunque playerId será null cuando el evento corresponda a toda la partida, como su inicio o finalización.

1.2. Relaciones entre eventos, partidas y jugadores

Cada evento pertenecerá a una única partida mediante matchId. Los participantes se identificarán al iniciar la partida y los eventos posteriores deberán hacer referencia a esos jugadores.

El identificador de un jugador se conservará entre partidas para permitir estadísticas acumuladas. Sin embargo, su color en ajedrez o su equipo en baloncesto se establecerá para cada participación.

Algunos eventos estarán relacionados directamente con otros:

- Una captura de ajedrez referenciará el movimiento que la produjo.

- Una asistencia o un rebote referenciará el tiro correspondiente.
- La entrada y salida de boxes compartirán un identificador de parada.
- Un fantasma consumido en Pac-Man referenciará la pastilla de poder que permitió consumirlo.

Estas relaciones permitirán realizar análisis que van más allá de contar acciones.

1.3. Eventos generales

Los tres juegos compartirán dos eventos generales:

Evento	Función
MATCH_STARTED	Registrar los participantes y la configuración inicial de la partida.
MATCH_FINISHED	Registrar el resultado y el motivo de finalización.

La información de estos eventos se adaptará al juego. En ajedrez se indicarán los colores y los relojes; en baloncesto, los equipos y períodos; en carreras, los pilotos y las vueltas previstas; y en Pac-Man, el jugador, el nivel inicial y las vidas disponibles. Al finalizar, se registrará el resultado de ajedrez, el marcador de baloncesto, la clasificación de la carrera o la puntuación obtenida y el motivo de finalización en Pac-Man.

.

2. Relaciones entre eventos y juegos

Se seleccionaron ajedrez, baloncesto, carreras y Pac-Man porque permiten analizar dinámicas diferentes. Ajedrez se organiza por turnos, baloncesto combina acciones individuales y resultados por equipo, y carreras permite observar tiempos y cambios de posición. Pac-Man permite analizar puntuaciones, niveles y secuencias de acciones durante un efecto temporal.

Cada juego tendrá un grupo pequeño de eventos, elegido según las estadísticas que se desea obtener.

2.1. Ajedrez

En ajedrez se analizarán los movimientos, las capturas y el uso del tiempo disponible.

Al comenzar la partida, se registrarán dos jugadores distintos: uno con blancas WHITE y otro con BLACK. También se definirán el tiempo inicial de sus relojes y el incremento por jugada.

Evento	Qué registra	Datos específicos
MOVE_PLAYED	Un movimiento completado.	moveNumber, piece, from, to, decisionTimeMs,

		clockBeforeMs, clockAfterMs.
PIECE_CAPTURED	Una captura realizada durante un movimiento.	moveEventId, capturedPiece, capturedPlayerId.

Los campos from y to indicarán la casilla de origen y destino. Por ejemplo, un movimiento de e2 a e4 permitirá identificar el recorrido de la pieza.

Para simplificar el procesamiento, moveNumber contará cada jugada individual: blancas realizarán la primera, negras la segunda y así sucesivamente. Esta numeración será propia del sistema y no la numeración tradicional de movimientos completos.

Cuando exista una captura, moveEventId la relacionará con el movimiento que la produjo. Si no hubo captura, solamente se registrará MOVE_PLAYED.

Los datos del reloj permitirán conocer cuánto tardó el jugador y cuánto tiempo tenía antes y después de mover, considerando el incremento configurado. Si agota su tiempo antes de completar una jugada, la partida terminará por tiempo, sin registrar ese movimiento como completado.

El evento final indicará al ganador, si lo hubo, y el motivo: jaque mate, abandono, tablas o tiempo agotado.

2.2. Baloncesto

En baloncesto se analizarán el rendimiento de los jugadores, la efectividad de tiro, las faltas y las rachas de anotación.

La simulación inicial no incluirá sustituciones. Además, se establecerá que un jugador queda fuera del partido al acumular cinco faltas personales.

Evento	Qué registra	Datos específicos
SHOT_ATTEMPTED	Un tiro de campo o tiro libre.	teamId, period, periodTimeRemainingMs, shotValue, made.
ASSIST_RECORDED	Una asistencia a una canasta.	teamId, period, scorerId, shotEventId.
REBOUND_COLLECTED	Un rebote obtenido.	teamId, period, reboundType, shotEventId.
TURNOVER_COMMITTED	Una pérdida de balón.	teamId, period, reason.
FOUL_COMMITTED	Una falta personal.	teamId, period, affectedPlayerId, reason.

En los tiros, shotValue indicará si el intento vale uno, dos o tres puntos, mientras made señalará si fue encestado. Los puntos se calcularán con esa información, sin agregar otro evento para la misma anotación.

Una asistencia deberá estar relacionada con una canasta de campo de un compañero. No podrá asociarse a un tiro libre, un intento fallado ni una canasta del propio jugador que da la asistencia.

Los rebotes se clasificarán como ofensivos o defensivos y se relacionarán con un tiro fallado. No todos los fallos tendrán un rebote, ya que puede terminar el período o quedar otro tiro libre pendiente.

Las faltas se acumularán por jugador. Al llegar a cinco, el sistema lo considerará excluido y no aceptará nuevas acciones suyas en esa partida. No será necesario crear otro evento para determinar esta condición.

Al finalizar, el marcador registrado deberá coincidir con los puntos calculados a partir de los tiros encestados.

2.3. Carreras

En carreras se analizarán los tiempos por vuelta, las posiciones, las paradas en boxes y las penalizaciones.

Para mantener controlado el alcance inicial, se simularán carreras en las que todos los pilotos completen las vueltas previstas.

Evento	Qué registra	Datos específicos
LAP_COMPLETED	Una vuelta completada.	lapNumber, lapTimeMs, position.
PIT_ENTERED	Una entrada a boxes.	stopId, lapNumber, reason.
PIT_EXITED	Una salida de boxes.	stopId, lapNumber, tireCompound.

PENALTY_APPLIED	Una sanción al piloto.	penaltyType, value, reason, lapNumber.

Las entradas y salidas de boxes compartirán stopId, para calcular cuánto duró cada parada. El tiempo en boxes ya estará incluido en el tiempo de vuelta, por lo que no se añadirá nuevamente al total.

Se utilizarán dos tipos de penalizaciones:

Tipo	Aplicación
TIME	Añadir al tiempo total los milisegundos indicados en value.
POSITION	Hacer retroceder al piloto la cantidad de puestos indicada en value.

Primero se ordenarán los pilotos por su tiempo total, incluyendo las sanciones de tiempo. Después se aplicarán las sanciones de posiciones según el orden de sequence. Los pilotos afectados por ese desplazamiento avanzarán, y ninguno podrá quedar por debajo del último puesto.

Si dos pilotos tienen el mismo tiempo ajustado, se utilizará su orden de llegada en la última vuelta para desempatar. Las sanciones no modificarán la mejor vuelta, porque se aplicarán al resultado final.

La clasificación de MATCH_FINISHED deberá coincidir con la obtenida mediante estos cálculos.

2.4. Pac-Man

En Pac-Man se analizarán las pastillas consumidas, los fantasmas capturados, las vidas perdidas y los niveles completados. Se simularán partidas de un solo jugador, sin representar todos los movimientos del laberinto.

Evento	Qué registra	Datos específicos
PELLET_EATEN	Una pastilla normal consumida.	level, pelletId.
POWER_UP_COLLECTED	Una pastilla de poder consumida.	level, pelletId, durationMs.
GHOST_EATEN	Un fantasma consumido durante un efecto de poder.	level, ghostId, powerUpEventId.
LIFE_LOST	Una vida perdida.	level, cause, livesRemaining.
LEVEL_COMPLETED	Un nivel completado.	level.

Para esta simulación, cada pastilla normal otorgará 10 puntos y cada pastilla de poder, 50. Los fantasmas consumidos durante un mismo efecto otorgarán 200, 400, 800 y 1600 puntos según su orden. No se incluirán frutas ni vidas adicionales.

powerUpEventId referenciará un evento POWER_UP_COLLECTED de la misma partida y jugador. Su efecto estará activo desde su timestamp hasta antes de timestamp + durationMs; una nueva pastilla de poder, LIFE_LOST o LEVEL_COMPLETED terminará el efecto anterior. La nueva pastilla reiniciará la secuencia de puntuación de fantasmas.

Cada pelletId será único dentro de su nivel. Durante un mismo efecto se permitirá consumir una sola vez cada uno de los cuatro ghostId: BLINKY, PINKY, INKY y CLYDE. Los niveles avanzarán de uno en uno y livesRemaining disminuirá en uno por cada LIFE_LOST. Al llegar a cero, solamente se aceptará MATCH_FINISHED.

3. Diseño de eventos y estadísticas

Los eventos se eligieron por la información que aportan al análisis. La propuesta busca registrar lo necesario para obtener resultados útiles, sin representar todas las acciones posibles de cada videojuego.

Las estadísticas permitirán consultar el desempeño de un jugador, revisar una partida y relacionar distintos eventos. Además, cada juego tendrá una regla que requiera observar una secuencia de acciones.

3.1. Estadísticas de ajedrez

Estadística	Cómo se obtiene
Movimientos por jugador	Contando sus eventos MOVE_PLAYED.
Capturas por jugador	Contando sus eventos PIECE_CAPTURED.
Tiempo promedio de decisión	Sumando sus tiempos de decisión y dividiendo entre sus movimientos.
Duración de la partida	Calculando la diferencia entre el inicio y el final.

Movimientos totales	Contando todos los movimientos de la partida.
Tiempo promedio en movimientos con captura	Relacionando las capturas con sus movimientos y promediando sus tiempos de decisión.
Porcentaje de victorias por color	Dividiendo las victorias del color entre las partidas completadas, incluidos los empates, y multiplicando por 100.

Se detectará presión de tiempo cuando un jugador complete tres jugadas consecutivas propias con menos de 60 segundos disponibles al iniciar cada una. Los movimientos del rival no interrumpirán esta secuencia.

Esta regla permitirá identificar una situación de tiempo reducido, pero no determinar si las decisiones fueron buenas o malas.

3.2. Estadísticas de baloncesto

Estadística	Cómo se obtiene
Puntos por jugador	Sumando el valor de sus tiros encestados.
Asistencias, rebotes, pérdidas y faltas	Contando los eventos correspondientes.

Marcador por equipo	Sumando los puntos de sus jugadores.
Efectividad de campo	Tiros de dos y tres puntos encestados ÷ intentados × 100.
Efectividad de triples	Triples encestados ÷ intentados × 100.
Efectividad de tiros libres	Tiros libres encestados ÷ intentados × 100.
Puntos de canastas asistidas	Relacionando las asistencias con los tiros y sumando los puntos de esas canastas.
Jugadores excluidos por faltas	Identificando a quienes alcanzaron cinco faltas personales.

Cuando no existan intentos en una categoría, la efectividad se guardará como null y se mostrará como “sin intentos”. Esto evitará confundir la falta de datos con una efectividad del cero por ciento.

Se detectará una racha de anotación cuando un equipo consiga al menos ocho puntos consecutivos sin que el rival anote. Un tiro fallado, una falta o una pérdida no cortará la racha; una anotación del equipo contrario sí lo hará.

También se identificará el momento en que un jugador alcance su quinta falta personal, para comprobar que no participe en acciones posteriores

3.3. Estadísticas de carreras

Estadística	Cómo se obtiene
Mejor vuelta por piloto	Seleccionando su menor tiempo de vuelta.
Tiempo promedio por vuelta	Sumando sus tiempos y dividiendo entre las vueltas completadas.
Tiempo total sin sanciones	Sumando sus tiempos de vuelta.
Cantidad de paradas	Contando las entradas y salidas de boxes correctamente relacionadas.
Duración de cada parada	Calculando la diferencia entre la salida y la entrada.
Penalizaciones por piloto	Contando las sanciones y acumulando sus valores por separado según el tipo.
Clasificación definitiva	Aplicando las penalizaciones al orden inicial por tiempo.
Cambio de clasificación	Comparando el puesto sin sanciones con el definitivo.
Vuelta más rápida de la carrera	Seleccionando el menor tiempo entre todos los pilotos.

Se detectará una remontada cuando un piloto termine una vuelta en la última posición y finalmente gane según la clasificación definitiva.

Las posiciones de sanción acumuladas y el cambio real de puesto se mostrarán por separado, porque no siempre serán iguales.

3.4. Estadísticas de Pac-Man

Estadística	Cómo se obtiene
Pastillas normales y de poder	Contando PELLET_EATEN y POWER_UP_COLLECTED por separado.
Fantasmas consumidos	Contando GHOST_EATEN por jugador y partida.
Puntuación total	Sumando los puntos de las pastillas y los fantasmas según su orden dentro de cada efecto.
Vidas perdidas y restantes	Contando LIFE_LOST y restando ese total de initialLives.
Niveles completados	Contando LEVEL_COMPLETED.
Duración de la partida	Calculando la diferencia entre MATCH_STARTED y MATCH_FINISHED.
Fantasmas por pastilla de poder	Agrupando GHOST_EATEN por powerUpEventId, incluyendo efectos sin capturas.

Puntuación acumulada y mejor puntuación por jugador	Sumando los resultados de sus partidas y seleccionando la mayor puntuación.

Se detectará una cadena de fantasmas cuando el jugador consuma al menos tres fantasmas distintos durante el efecto de una misma pastilla de poder. Los eventos deberán compartir powerUpEventId y ocurrir antes de que el efecto expire o sea interrumpido. Se registrará una sola detección por efecto, aunque se consuma un cuarto fantasma.

3.5. Datos de prueba

Se prepararán al menos tres partidas de cada juego. Algunos jugadores aparecerán en varias partidas para comprobar los resultados acumulados.

También se enviarán eventos intercalados de partidas de diferentes juegos, con tiempos superpuestos, para demostrar que el sistema puede procesarlas en paralelo.

Juego	Casos que se probarán
Ajedrez	Victorias de ambos colores, empate, capturas y situaciones con y sin presión de tiempo.
Baloncesto	Tiros de distintos valores, aciertos, fallos, asistencias, rebotes, pérdidas, quinta falta y rachas de anotación.

Carreras	Tiempos variados, paradas, sanciones de tiempo y posiciones, y situaciones con y sin remontada.
Pac-Man	Pastillas normales y de poder, efectos sin fantasmas, cadenas de tres y cuatro fantasmas, expiración del poder, pérdida de vidas y avance de niveles.

Cada conjunto tendrá resultados esperados calculados previamente. Por ejemplo, seis tiros de campo encestados de diez intentos deberán producir una efectividad del 60 %.

Se prepararán, por separado, eventos inválidos para comprobar el manejo de errores: campos faltantes, eventos duplicados, referencias inexistentes, acciones sin cumplir las condiciones necesarias, secuencias de eventos incorrectas, valores inconsistentes y acciones realizadas después de una expulsión o de agotar las vidas disponibles.

3.6. Organización del procesamiento

Go se encargará de recibir los eventos y validar que contengan los campos necesarios y los valores permitidos. También identificará eventos duplicados y organizará la información por juego y partida.

Scala se encargará del análisis. Utilizará operaciones funcionales para filtrar eventos, transformar los datos, agruparlos por jugador o partida y combinar la información relacionada. A partir de estas operaciones, calculará estadísticas como la efectividad de tiro, la mejor vuelta y el tiempo promedio de decisión.

También analizará secuencias de eventos para detectar situaciones como presión de tiempo en ajedrez, rachas de anotación en baloncesto y remontadas en carreras. Finalmente, generará resultados en formato JSON para que puedan consultarse, almacenarse o presentarse.